Assignment 2

20201559 김민준

I. Complete the dataset class

I implemented IMDbData class as fallow:

1. __len__ method

```
class IMDbData:
  def __init__(self, path_list):
    self.paths = path_list
    self.tokenizer = Tokenizer()

  def __len__(self) -> int:
    """..."""
    return len(self.paths)

  def __getitem__(self, idx:int) -> Tuple[List[str], int]:
    """..."""
    path = self.paths[idx]
    label = 1 if 'pos' in str(path) else 0
    tokens = self.tokenizer(read_txt(path))
    return tokens, label
```

__len__ method are used for length of the path_list, so I return len(self.apths).

2. __getitem__ method

__geitem__ method are used for access specific index of tokens and label, so I implemented like this, but this codes are inefficient, because it have to read text file at every access. So if VRAM is large then we just load the text file in the IMDbData class for caching.

II. Complete String to idx Converter

1. __init__ method

```
class Str2Idx2Str:
  def __init__(self, vocab: List[str]):
    '''...
    self.idx2str = vocab
    self.str2idx = {}
    for i, s in enumerate(vocab):
      self.str2idx[s] = i
```

I have to implement 2 fields idx2str, str2idx.

i) idx2str

I just store the vocab variable in idx2str, because it's already list, and it can access list for indexing

ii) str2idx

I declare str2idx as directory, key : str, value : idx, and I use for loop to complete this directory.

iii) unknown_idx and "UNKNOWN" string

```
'''
You have to add these lines,
And explain in your report what is the function of these two lines
'''
self.unknown_idx = len(self.str2idx)
self.idx2str.append("UNKNOWN")
```

index or string can be not in the vocab, so i use 'unkown_idx' and string "UNKOWN" at this situation. so if someone want to convert unknown words to idx, then it will be return unknown_idx. At the opsite direction, someone wand to convert unknown_idx to string, then this function returns "UNKOWN"

2. __call__ method

```
# Write your code from here
if isinstance(alist[0], list):
  return [self.__call__(i) for i in alist]
elif isinstance(alist[0], str):
  return [self.str2idx[word] if word in self.str2idx else self.unknown_idx for word in alist]
elif isinstance(alist[0], int):
  return [self.idx2str[idx] for idx in alist]
else:
  raise ValueError(f"Invalid input type: {type(alist)}")
```

At the comments, argument 'alist' are Union type of List][str], List[int], List[List], but at the type definition of __call__ functions, it includes int, str too. But I implemented this method following comments.

If alist instance are List[List] then it recursively called __call__ function for each element(list), else if it was List[str], then it use str2idx dictional for convert to index, if word are not in dictionary's key, then it returns unknown_idx. (refer I-1-iii). and if the alist are List[int], just use idx2str to convert index to string. and If the alist's type is invalid then raise error.

III. Complete Collate Function

```
class PackCollateWithConverter:
  def __init__(self, converter: Str2Idx2Str):
    self.converter = converter

  def __call__(self, batch: List[Tuple[List[str], int]]):
    '''...

    # Write your code from here
    word_sentences_in_idxs = [th.LongTensor(self.converter(batch[i][0])) for i in range(len(batch))]
    label_tensor = th.FloatTensor([batch[i][1] for i in range(len(batch))])
```

__call__ method is used for generating torch tensor. Argument of this method Is batch is List of (List[str], int) tuple.

And the dtype of each tensor are long and float. So I implement using list comprehension and access each element to index.

IV. Implement Binary Cross Entropy Loss

```python
def get_binary_cross_entropy_loss(pred:torch.FloatTensor, target:torch.FloatTensor, eps=1e-8) -> torch.FloatTensor:
  '''...
  return th.mean(-(target * th.log(pred + eps) + (1 - target) * th.log(1 - pred + eps)))
```

I use this formular for BCE, and use th.mean and th.log function for Implement.

$$L_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(Y_i \cdot \log \hat{Y}_i + (1 - Y_i) \cdot \log (1 - \hat{Y}_i))$$

V. Complete Training Loop

i) _get_accuracy

This method returns accuracy for given prediction(model output) and target (label). it determine pred is larger than threshold or not. It is the binary classification results of model.

Use .float() method because At III. I use FloatTensor to store the label of dataset. And it compare target and pred value to determine its collect or not.

according to comments, return value might be float. So I use .float().mean() to average all of batch, and use .item() to convert FloatTensor to float.

```python
def _get_accuracy(self, pred, target, threshold=0.5):
  '''...
  # return value is float
  pred = (pred > threshold).float()
  return (pred == target).float().mean().item()
```

ii) _get_loss_and_acc_from_single_batch

```python
def _get_loss_and_acc_from_single_batch(self, batch):
  '''...
  batch = (batch[0].to(self.device), batch[1].to(self.device))
  pred = self.model(batch[0])
  loss = self.loss_fn(pred, batch[1])
  acc = self._get_accuracy(pred, batch[1])
  return loss, acc
```

At __init__method of this class, move model to device(cuda). So, i have to move batch to device to use model.

This method calculates the loss and acc using loss_fn and _get_accuracy.

according to comment. dtype of Loss and acc might be tensor, float. but I already convert acc to float at _get_accuracy methods. I don't have to consider data dype.

iii) _train_by_single_batch

```python
def _train_by_single_batch(self, batch):
  '''...
  loss, acc = self._get_loss_and_acc_from_single_batch(batch)
  self.optimizer.zero_grad()
  loss.backward()
  self.optimizer.step()
  return loss.item(), acc
```

I use _get_loss_and_acc_from_single_batch function to get loss and acc. I set optimizer's gradiants to zero to remove previous values. And do backward propagation and step of optimizer. (this will update the weight of the model). And return the loss and acc.
according to comments. dtype of the return value might be float. so, I use the .item() method to convert FloatTensor to float.

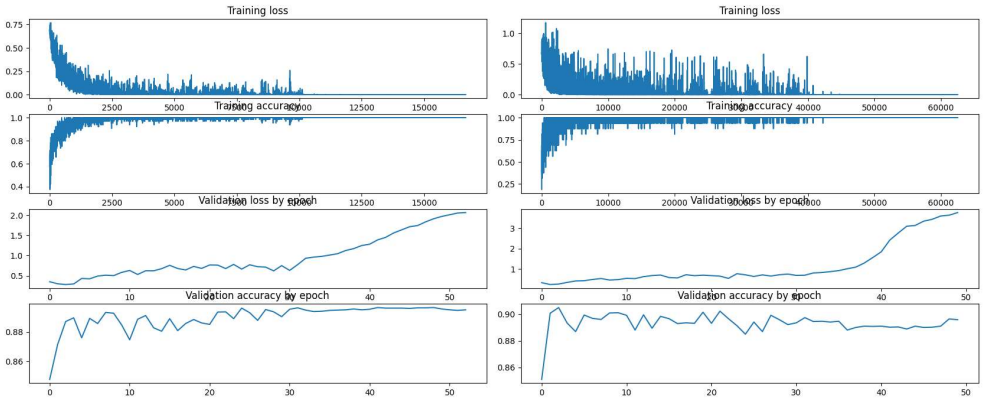
iv) validate

```
def validate(self, external_loader=None):
    '''...
    ### Don't change this part
    if external_loader and isinstance(external_loader, DataLoader):...
    else:...
    self.model.eval()
    '''
    Write your code from here, using loader, self.model, self.loss_fn.
    '''
    total_loss = 0
    total_acc = 0
    with torch.no_grad():
        for batch in loader:
            loss, acc = self._get_loss_and_acc_from_single_batch(batch)
            total_loss += loss.item()
            total_acc += acc
    return total_loss / len(loader), total_acc / len(loader)
```

validate method will calculate loss and acc for given data loader. so, it is used at the validation step or test step.
I used torch.no_grad() to diable caculating gradients of the model to increase the speed.
I cumulative loss and acc at the total_* variable. and devide to the length of the loader to calculate the Mean value.

v) Results

I tried to change the batch size of the data loader (train, valid, test). But I think the train set are too small so model was overfitted in the trainset. after 20 iterations (Regardless the batch size), train loss is zero and train accuracy is 1. So the model can't improve anymore. And at this point, validation accuracy is 90%. And I think it is the limits of this assignment. This is my training result, left is batch size 64, and right is batch size 16.



VI. Analyze the Prediction of the model

1. Main Criteria

I think if 't word is in the sentence, then the model predicts the sentence is negative.

```
your_text = """
This movie is terrific
"""
your_text=your_text.strip()
for i in range (5):
  output =estimate_sentiment_of_given_txt(model, your_text).item()
  pred = "pos" if output > 0.5 else "neg"
  print(f"""
  text : {your_text.strip()}
  pred : {pred}
  score: {f"{output:.4f}"}""")
  your_text = your_text + "'t"
```

✓ 0.0s

```
text : This movie is terrific
pred : pos
score: 0.8589

text : This movie is terrific't
pred : neg
score: 0.2015

text : This movie is terrific't't
pred : neg
score: 0.1080

text : This movie is terrific't't't
pred : neg
score: 0.0722

text : This movie is terrific't't't't
pred : neg
score: 0.0549
```

If I add 's word then score is decrease but it remains positive.

```
text : This movie is terrific
pred : pos
score: 0.8589

text : This movie is terrific's
pred : pos
score: 0.7493

text : This movie is terrific's's
pred : pos
score: 0.7366

text : This movie is terrific's's's
pred : pos
score: 0.7395

text : This movie is terrific's's's's
pred : pos
score: 0.7461
```

I think words that directly express feelings, like 't, bad, good, happy are a very important feature of the model's output.

2. Mistake

```
text : i will recommand this movie to everyone.
pred : pos
score: 0.8503
```

```
text : i will not recommand this movie to anyone.
pred : pos
score: 0.5772
```

```
text : i won't recommand this movie to anyone.
pred : neg
score: 0.4930
```

I think this model's performance is not that good. Second results might be negative review. But this model's prediction is positive. But I mentioned earlier, if I use 't word then it becomes negative.

3. Input to text

I try to get some reviews at the IMDB website. But if I use old movies they might exist at the dataset. so I selected mickey 17, from this link: https://www.imdb.com/title/tt12299608/reviews/?ref_=tt_ururv_sm

I think my model is very good at predicting negative or positive. (it almost predicts the star ratings of the review)

4. Analysis

```
'''
This will print out top-k most incorrect prediction on test set
'''
texts = print_top_k_loss_case(test_loader, sorted_indices, entire_loss, entire_pred, k=100, descending=True)
```

✓ 0.0s

```
Average Number of token in text: 213.92
Number of positive samples: 28
Number of UNKNOWN tokens in positive samples:  0.8929
Number of negative samples: 72
Number of UNKNOWN tokens in negative samples:  1.4304
```

```
'''
This will print out top-k most correct prediction on test set
'''

texts = print_top_k_loss_case(test_loader, sorted_indices, entire_loss, entire_pred, k=100, descending=False)
```

✓ 0.0s

```
Average Number of token in text: 174.73
Number of positive samples: 80
Number of UNKNOWN tokens in positive samples:  1.3621
Number of negative samples: 20
Number of UNKNOWN tokens in negative samples:  0.1316
```

I think we can find 3 features in this result.

i) Length of sentence

We can check if the sentence is long then the model's accuracy is bad.

ii) Bias of model

At the incorrect part, there are lots of negative samples compared to positive, opposite direction, there are lots of positive samples in correct parts. It implies model is biased on the positive part.

iii) UNKNOWN token

If UNKOWN token is in the negative sentence. Then model's accuracy is bad.